

06

Módulo 06 - Tipos de aprendizagem - aprendizagem supervisionada

Fundamentos de Inteligência Artificial (IA)

Resumo do Módulo

- ◆Previsão/Estimação de uma Variável a Partir de um Conjunto de Outras Variáveis.
- ◆Definição e Exemplos Práticos.
- ◆Variáveis Qualitativas vs. Variáveis Quantitativas.
- ◆Algoritmos Comuns: Regressão Linear, Árvores de Decisão, SVM, Entre Outros.
- ◆Lab

Machine Learning

◆ Supervisionada (Supervised):

- Regressão (Regression): **Linear Simples** • Linear Múltipla • Polinomial • Lasso • Ridge
- Classificação (Classification): **Regressão Logística** • KNN • SVM • Kernel SVM • Naïve Bayes
- Séries Temporais (Time Series): **ARIMA** • SARIMA • LSTM • Prophet • Suavização Exponencial
- Métodos de Conjunto (Ensemble Methods): **Árvore de Decisão** • Random Forest • XGBoost • CatBoost
- Aprendizagem Profunda (Deep Learning): **Redes Neurais Artificiais** • Convolucionais • Recorrentes • **Transformers + Atenção**

◆ Não Supervisionada (Unsupervised):

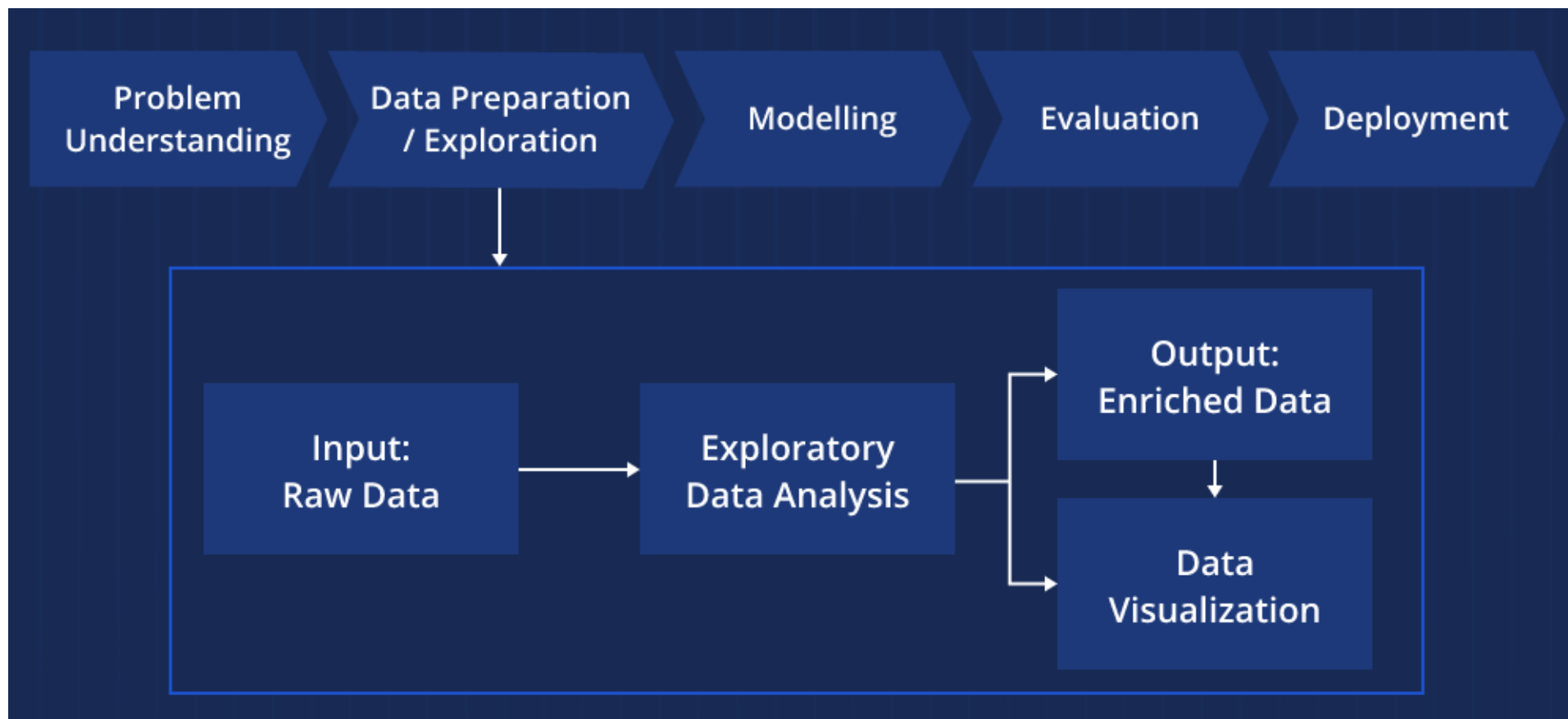
- ~~Clustering (Agrupamento): **K-means** • Hierárquico • GMM~~
- ~~Redução de Dimensionalidade (Dimensionality Reduction): **PCA** • Kernel PCA • LDA~~

◆ Reforço (Reinforcement):

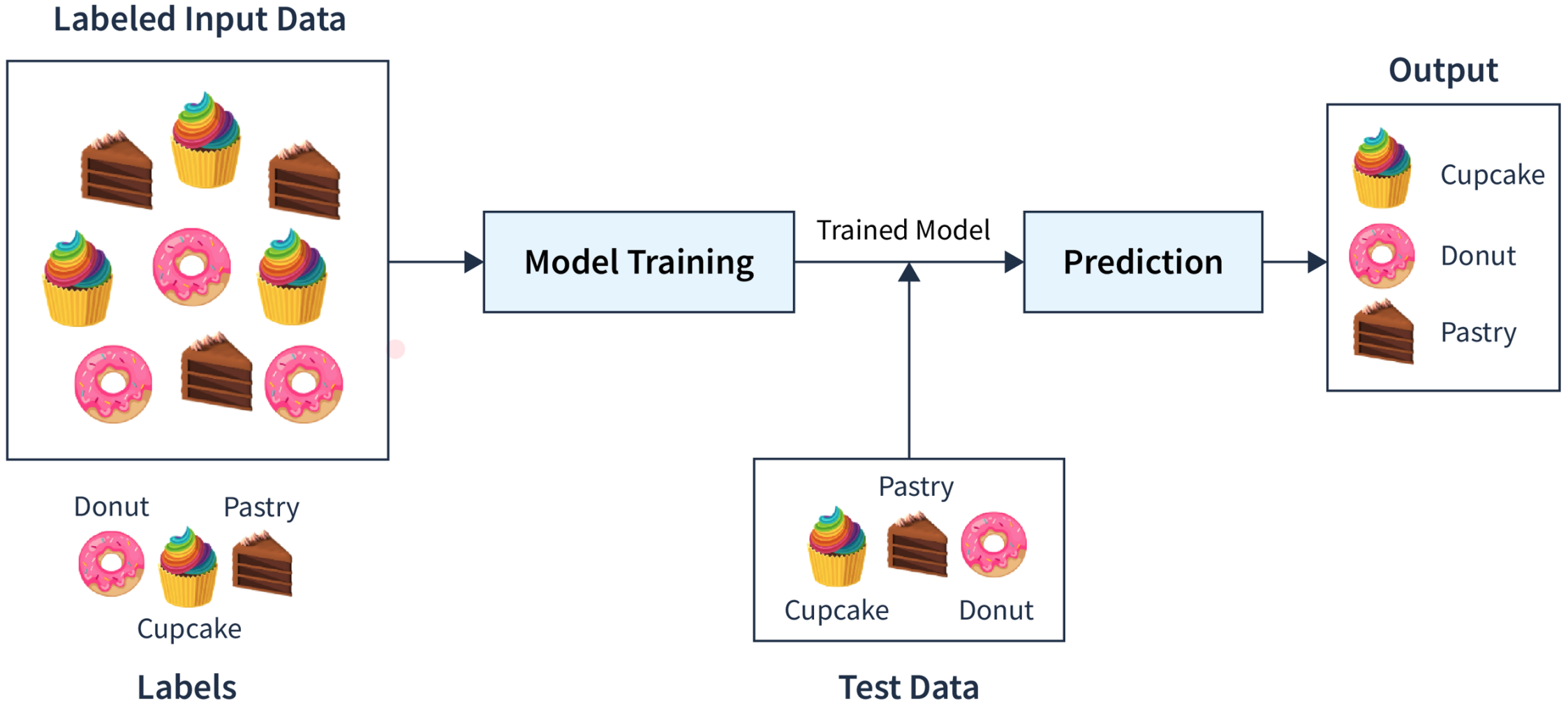
- ~~Algoritmos de Reforço (Reinforcement Algorithms): **Limite de Confiança Superior** • Amostragem de Thompson • **Q-Learning** • Deep Q-Learning • PPO~~
- ~~Algoritmos Genéticos (Genetic Algorithms): **Algoritmo Genético Clássico** • NSGA-II • Algoritmos Meméticos~~

Exploratory data analysis (EDA)

Análise Exploratória de Dados (EDA) é a abordagem inicial para examinar conjuntos de dados, resumindo as suas principais características através de métodos visuais e estatísticos.



Aprendizagem supervisionada (Supervised Machine Learning)

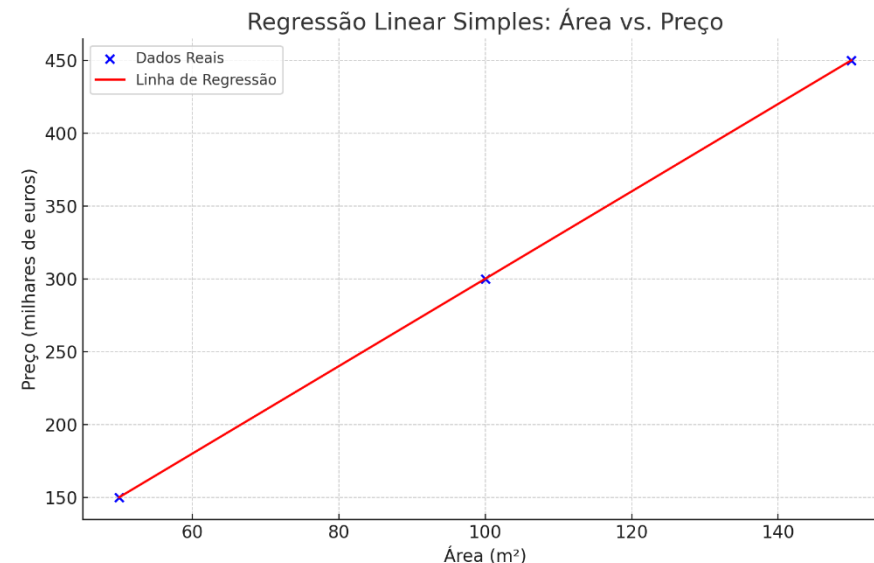


Comparar algoritmos de Regressão (Regression)

Algoritmo	Prós	Contras
Linear Simples	Fácil de interpretar; rápido de treinar	Funciona apenas para relações lineares simples
Linear Múltipla	Captura múltiplas variáveis independentes	Suscetível ao sobreajuste se houver muitas variáveis
Polinomial	Modela relações complexas e não-lineares	Pode ser difícil de generalizar; alta complexidade
Lasso	Reduz coeficientes a zero, facilitando a interpretação	Não lida bem com correlações entre variáveis
Ridge	Reduz a magnitude dos coeficientes, evitando sobreajuste	Mantém todos os coeficientes, mesmo os menos relevantes

Regressão Linear Simples (Linear Regression)

- ◆ Modela a relação entre uma **variável independente (X)** e uma **variável dependente (Y)**.
- ◆ Usada para prever valores de Y com base em X.



```
from sklearn.linear_model import LinearRegression

# Dados: Área (m²) e Preço (milhares de euros)
X = [[50], [100], [150]] # Área
y = [150, 300, 450]      # Preço

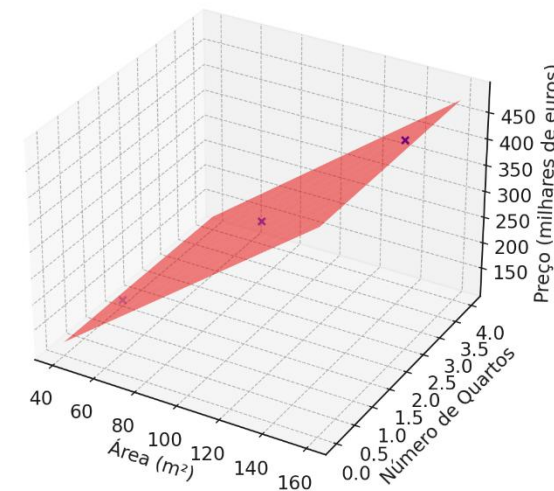
# Criar e ajustar o modelo
modelo = LinearRegression().fit(X, y)

# Previsão para uma área de 120 m²
previsao = modelo.predict([[120]])
print(f"Preço previsto: {previsao[0]:.2f} milhares de euros")
```

Regressão Linear Múltipla (Multiple Linear Regression)

Regressão Linear Múltipla: Área e Número de Quartos vs. Preço

- ♦ Extensão da regressão linear simples e modela a relação entre duas ou mais **variáveis independente** (X) e uma **variável dependente** (Y).
- ♦ Usada para modelar a relação entre múltiplos fatores



```
from sklearn.linear_model import LinearRegression

# Dados: Área (m²), Número de Quartos e Preço (milhares de euros)
X = [[50, 1], [100, 2], [150, 3]] # Área e Número de Quartos
y = [150, 300, 450]               # Preço

# Criar e ajustar o modelo
modelo = LinearRegression().fit(X, y)

# Previsão para uma casa de 120 m² com 3 quartos
previsao = modelo.predict([[120, 3]])
print(f"Preço previsto: {previsao[0]:.2f} milhares de euros")
```

Regressão Polinomial (Polynomial Regression)

- ◆ Extensão da regressão linear simples que usa uma função polinomial para modelar a relação entre (X) e (Y).

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
# Dados: Área (m²) e Preço (milhares de euros)
```

```
X = [[50], [100], [150]] # Área
y = [150, 300, 500]      # Preço
```

```
# Criar características polinomiais (grau 2)
```

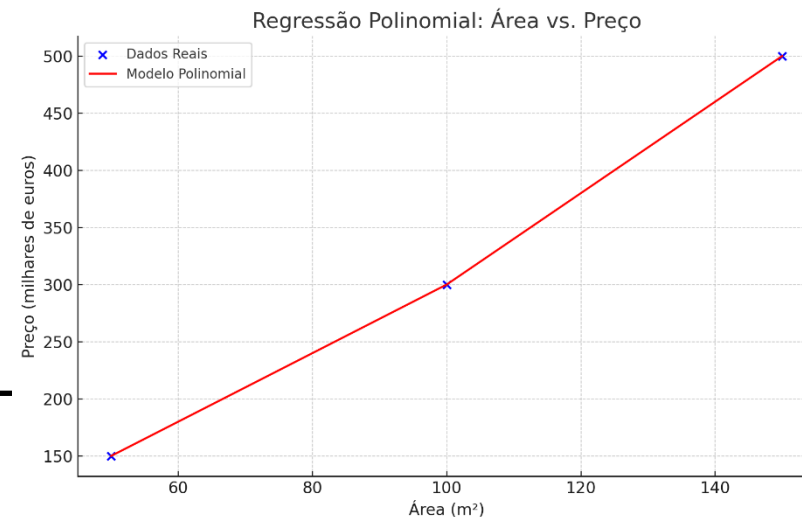
```
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```

```
# Criar e ajustar o modelo
```

```
modelo_polinomial = LinearRegression().fit(X_poly, y)
```

```
# Previsão para uma área de 120 m²
```

```
previsao = modelo_polinomial.predict(poly.transform([[120]]))
print(f"Preço previsto: {previsao[0]:.2f} milhares de euros")
```



- ◆ Usada quando a relação entre as variáveis não é linear e requer um ajuste mais complexo

Lasso e Ridge (Lasso and Ridge Regression)

◆ Lasso (Least Absolute Shrinkage and Selection Operator):

- Aplica penalização que pode zerar coeficientes menos relevantes, simplificando o modelo.

◆ Ridge:

- Penaliza os coeficientes, reduzindo sua magnitude para evitar sobreajuste.

```
from sklearn.linear_model import LinearRegression, Lasso, Ridge
```

```
# Dados: Área (m²) e Preço (milhares de euros)
X = [[50], [100], [150]] # Área
y = [150, 300, 500]      # Preço
```

```
# Modelo Linear Simples (Standard)
modelo_linear = LinearRegression().fit(X, y)
```

```
# Modelos Lasso e Ridge
```

```
# O parâmetro alpha controla a força da regularização:
```

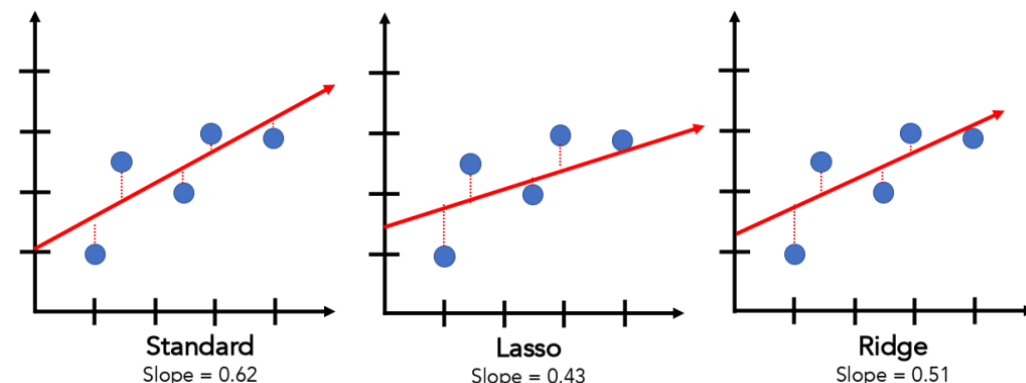
```
# - Quanto maior o alpha, maior a penalização aplicada aos coeficientes.
```

```
# - Para Lasso, um alpha elevado reduzir coeficientes a zero (útil para simplificar o modelo).
```

```
# - Para Ridge, um alpha elevado suaviza os coeficientes, reduzindo a magnitude deles.
```

```
modelo_lasso = Lasso(alpha=10.0).fit(X, y)
```

```
modelo_ridge = Ridge(alpha=10.0).fit(X, y)
```



- ◆ Lab 06 - Tipos de aprendizagem - aprendizagem supervisionada
 - 6.1. - Regressão (Regression) — Estimar a Duração de Voo

Comparar algoritmos de Classificação (Classification)

Algoritmo	Prós	Contras
Regressão Logística	Fácil de interpretar; bom para dados lineares	Funciona apenas para relações lineares simples
KNN (K-Nearest Neighbors)	Simples de entender; não requer treino	Suscetível ao sobreajuste se houver muitas variáveis
SVM (Support Vector Machine)	Bom para dados lineares com margens claras	Pode ser difícil de generalizar; alta complexidade
Kernel SVM	Captura relações não-lineares complexas	Não lida bem com correlações entre variáveis
Naïve Bayes	Rápido; bom para dados de alta dimensionalidade	Mantém todos os coeficientes, mesmo os menos relevantes

Regressão Logística (Logistic Regression)

A **Regressão Logística** usa a função sigmoide para prever valores entre 0 e 1, classificando em categorias como "Sim/Não" ou "Verdadeiro/Falso"

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
# Carregar dados e dividir em treino e teste
```

```
X, y = load_iris(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Treinar o modelo de Regressão Logística
```

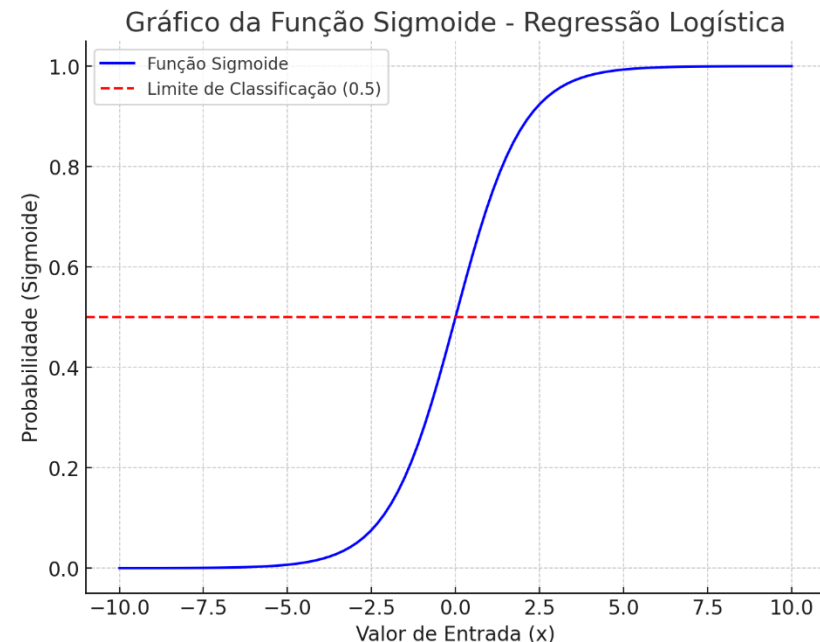
```
modelo = LogisticRegression()
```

```
modelo.fit(X_train, y_train)
```

```
# Previsão e avaliação
```

```
accuracy = modelo.score(X_test, y_test)
```

```
print(f"Acurácia do Modelo: {accuracy:.2f}")
```



K-Nearest Neighbors (KNN)

O **KNN** classifica um ponto olhando para os **K** vizinhos mais próximos e atribuindo a classe mais comum ou a média dos valores.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
# Carregar dados e dividir em treino e teste
```

```
X, y = load_iris(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Treinar o modelo KNN
```

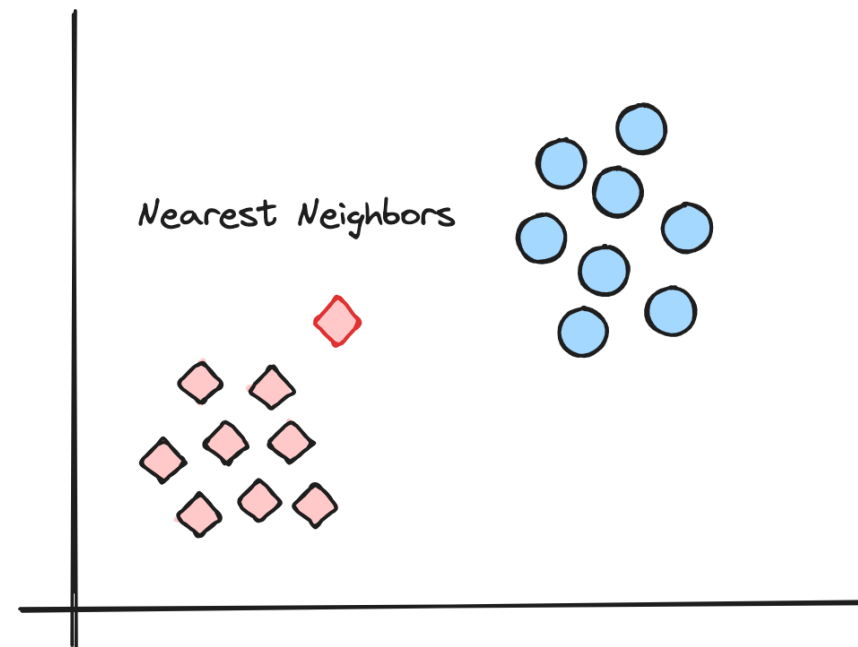
```
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X_train, y_train)
```

```
# Previsão e avaliação
```

```
accuracy = knn.score(X_test, y_test)
```

```
print(f"Acurácia do Modelo: {accuracy:.2f}")
```



Support Vector Machine (SVM)

SVM é um algoritmo de classificação que busca encontrar um **hiperplano** ótimo que **separa os dados** em duas ou mais classes

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
```

```
# Carregar dados e dividir em treino e teste
```

```
X, y = load_iris(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Treinar o modelo SVM
```

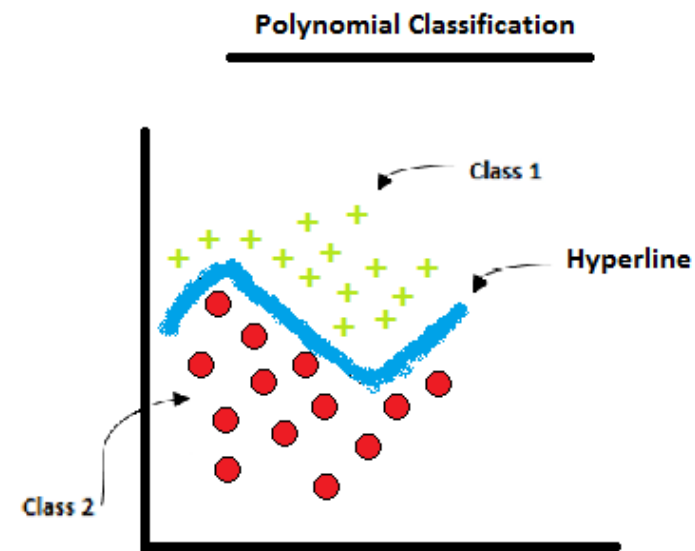
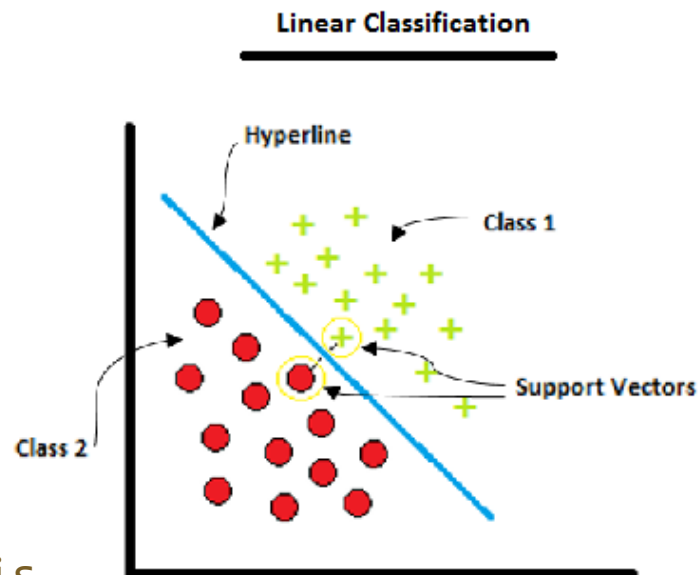
```
svm_model = SVC(kernel='linear')
```

```
svm_model.fit(X_train, y_train)
```

```
# Previsão e avaliação
```

```
accuracy = svm_model.score(X_test, y_test)
```

```
print(f"Acurácia do Modelo: {accuracy:.2f}")
```



Kernel Support Vector Machine (Kernel SVM)

Kernel SVM é uma extensão do SVM que utiliza funções de **kernel** para lidar com relações não-lineares nos dados.

kernel é uma **função matemática** usada para resolver problemas de **separação** de dados complicados

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# Carregar dados e dividir em treino e teste
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Treinar o modelo Kernel SVM
svm_model = SVC(kernel='rbf')
svm_model.fit(X_train, y_train)

# Previsão e avaliação
accuracy = svm_model.score(X_test, y_test)
print(f"Acurácia do Modelo: {accuracy:.2f}")
```

Naïve Bayes

Baseado no **Teorema de Bayes**, que calcula a probabilidade de cada classe.

- Assume que as **variáveis de entrada são independentes** (daí "**naïve**", ou **ingênuo**)

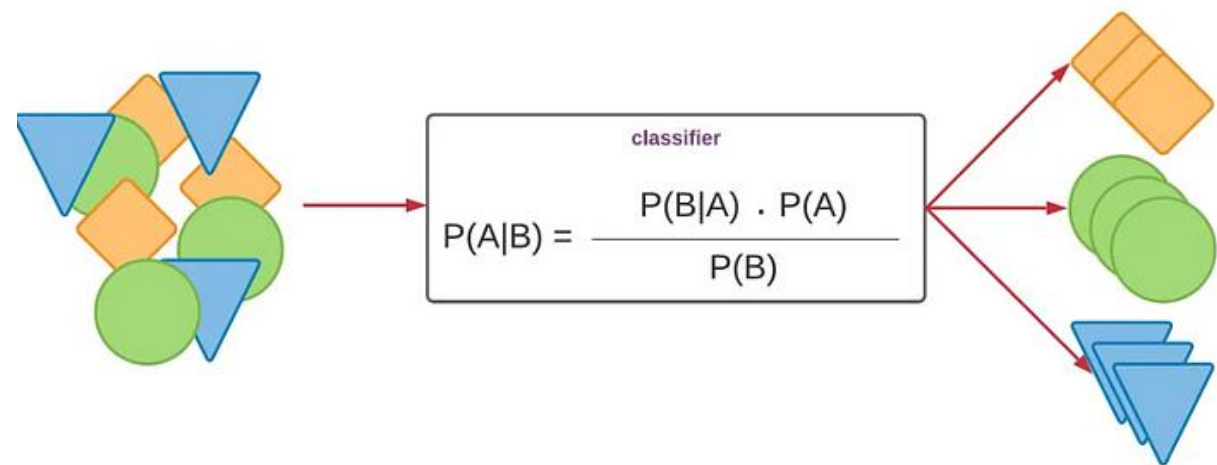
```
from sklearn.naive_bayes import GaussianNB
import numpy as np

# Dados de exemplo: features (cor) e rótulos (forma)
# 0 = não azul, 1 = azul
# 0 = não triângulo, 1 = triângulo
X = np.array([[1], [1], [0], [0], [1], [0], [0], [0]]) # Cores (1 = azul)
y = np.array([1, 1, 0, 0, 1, 0, 0, 0]) # Formas (1 = triângulo)

# Treinar o modelo de Bayes
model = GaussianNB()
model.fit(X, y)

# Probabilidade de ser triângulo dado que é azul
# 1 = azul
predicted_prob = model.predict_proba([[1]])

# Probabilidade de ser triângulo
print(predicted_prob[0][1])
```



Probabilidade de ser um triângulo sabendo que é azul = $\frac{\text{Probabilidade de ser azul se for um triângulo} \times \text{Quantos triângulos existem}}{\text{Probabilidade geral de ser azul}}$

- ◆ Lab 06 - Tipos de aprendizagem - aprendizagem supervisionada
 - 6.2. - Classificação (Classification) — Prever o Risco de Incidentes em Voos

Comparar algoritmos de Séries Temporais (Time Series)

Algoritmo	Prós	Contras
ARIMA	Bom para séries estacionárias; fácil de interpretar	Desempenho ruim para sazonalidades ou não-estacionariedade complexa
SARIMA	Lida com sazonalidade; extensão do ARIMA	Complexo de ajustar; requer identificação de múltiplos parâmetros
LSTM	Lida bem com dependências de longo prazo; útil para dados complexos	Requer grandes quantidades de dados e recursos computacionais significativos
Prophet	Fácil de usar; bom para sazonalidades e tendências	Pode ser impreciso com padrões não regulares; não lida bem com ruídos extremos
Suavização Exponencial	Simples de implementar; bom para séries com padrões simples	Desempenho fraco em séries complexas ou com múltiplas sazonalidades

ARIMA (Autoregressive Integrated Moving Average)

1. Componentes do ARIMA:

AR (Autoregressivo): Relação entre valores passados e presentes.

I (Integrado): Diferença entre observações para torná-las estacionárias.

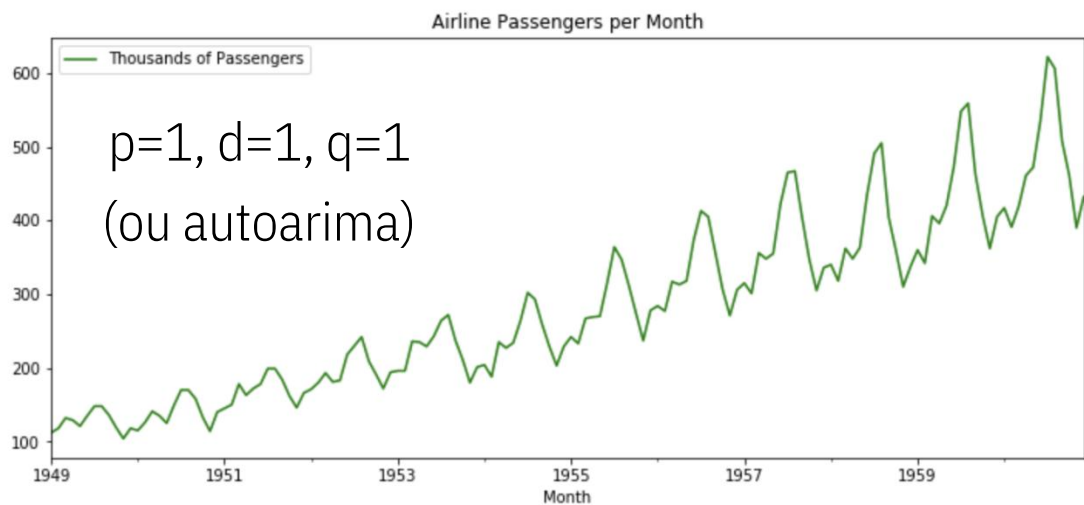
MA (Média Móvel): Erros passados influenciam os valores atuais.

2. Parâmetros:

p: Número de lags (valores anteriores) no modelo AR.

d: Diferenciações necessárias para estacionaridade.

q: Número de termos de média móvel.



```
# pip install statsmodels

import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

# Carregar dados de exemplo
data = pd.read_csv('dados_temporais.csv',
                   index_col='Data', parse_dates=True)

# Definir o modelo ARIMA
model = ARIMA(data['valor'], order=(p, d, q))

# Ajustar o modelo
model_fit = model.fit()

# Fazer previsão
forecast = model_fit.forecast(steps=10)
print(forecast)
```

SARIMA (Seasonal ARIMA)

```
# pip install statsmodels

import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Carregar dados de exemplo
data = pd.read_csv('dados_temporais.csv', index_col='Data', parse_dates=True)

# Definir o modelo SARIMA
model = SARIMAX(data['valor'], order=(p, d, q), seasonal_order=(P, D, Q, m))

# Ajustar o modelo
model_fit = model.fit()

# Fazer previsão
forecast = model_fit.forecast(steps=10)
print(forecast)
```

2. Parâmetros:

p: Número de lags (valores anteriores) no modelo AR.

d: Diferenciações necessárias para estacionaridade.

q: Número de termos de média móvel.

m: Período da sazonalidade (por exemplo, 12 para dados mensais).

1. Componentes do ARIMA:

AR (Autoregressivo): Relação entre valores passados e presentes.

I (Integrado): Diferença entre observações para torná-las estacionárias.

MA (Média Móvel): Erros passados influenciam os valores atuais.

Sazonalidade (S): Captura padrões que se repetem em intervalos fixos (como anual, mensal, semanal).

Só SARIMA

Rede Neural Recorrente (**RNN**) especialmente adequada para dados sequenciais

```
# pip install tensorflow
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

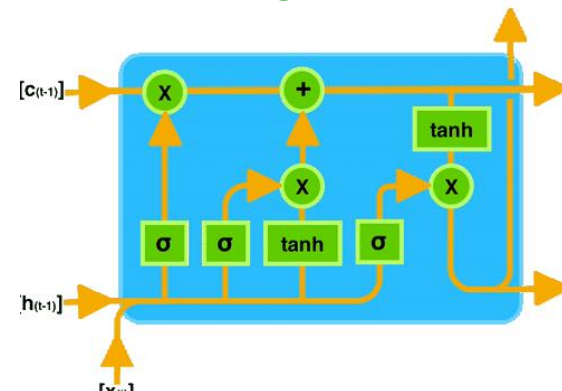
# Dados de exemplo
X = np.random.random((100, 10, 1)) # 100 amostras, 10 passos temporais, 1 feature
y = np.random.random((100, 1))

# Definir o modelo LSTM com ativação tanh
model = Sequential()
model.add(LSTM(50, activation='tanh', input_shape=(10, 1))) # Ativação alterada para tanh
model.add(Dense(1))

# Compilar o modelo com o otimizador Adam (ajusta a taxa de aprendizagem de forma adaptativa)
model.compile(optimizer='adam', loss='mse')

# Treinar o modelo
model.fit(X, y, epochs=200, verbose=0)

# Fazer previsão
forecast = model.predict(X[:1])
print(forecast)
```



Prophet (desenvolvido pelo Facebook)

Fácil de usar e otimizado para séries temporais com componentes sazonais e eventos irregulares.

- Capaz de lidar com grande volume de dados com **parâmetros simples** para ajuste fino.

```
# pip install prophet
```

```
import pandas as pd
from prophet import Prophet
```

```
# Carregar dados de exemplo
```

```
data = pd.read_csv('dados_temporais.csv')
```

```
data.columns = ['ds', 'y'] # Renomear colunas: 'ds' para data e 'y' para valor
```

```
# Definir e ajustar o modelo Prophet
```

```
model = Prophet()
```

```
model.fit(data)
```

```
# Fazer previsão
```

```
future = model.make_future_dataframe(periods=30) # Previsão para 30 dias
```

```
forecast = model.predict(future)
```

```
print(forecast[['ds', 'yhat']]) # yhat = valor previsto
```

Suavização Exponencial (Exponential Smoothing)

Método de previsão que dá **mais peso aos valores recentes**.

Bom para séries temporais de curto prazo.

Tipos de Suavização Exponencial:

- Simples (**SES**): Sem tendência ou sazonalidade
- Dupla (**DES**): Com tendência (como crescimento ou declínio)
- Tripla (**TES**): Tendência + sazonalidade

```
# pip install statsmodels
```

```
import pandas as pd
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
```

```
# Carregar dados de exemplo
data = pd.read_csv('dados_temporais.csv')
series = data['valor']
```

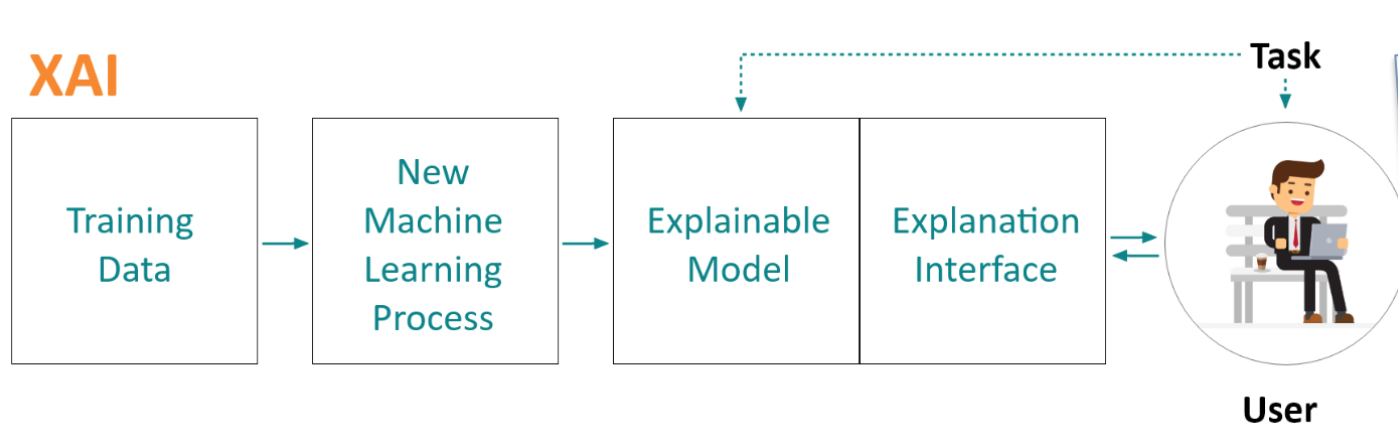
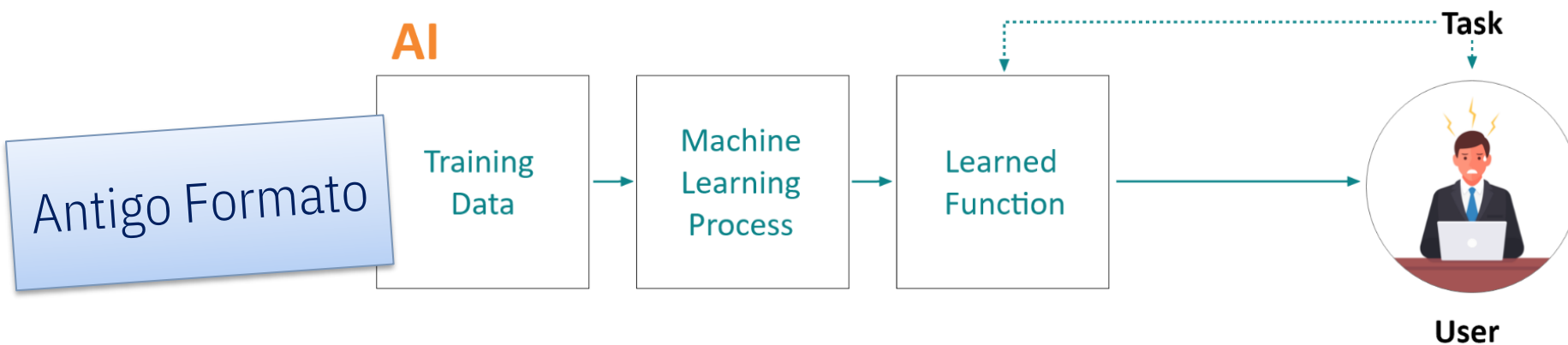
```
# Ajustar o modelo de Suavização Exponencial Simples
model = SimpleExpSmoothing(series).fit(smoothing_level=0.2, optimized=False)
```

```
# Fazer previsão
forecast = model.forecast(10) # Previsão para 10 períodos
print(forecast)
```

- ◆ Lab 06 - Tipos de aprendizagem - aprendizagem supervisionada
 - 6.3. - Séries Temporais (Time Series) — Previsão de procura e consumo

Explicabilidade (XAI - Explainable AI)

Inteligência Artificial Explicável (XAI) refere-se a sistemas de IA que conseguem fornecer explicações claras e compreensíveis sobre as decisões e resultados.



Nova tendência
(obtida da IA Simbólica
e muito por causa das
alucinações dos LLMS)

Comparar algoritmos de Métodos de Conjunto (Ensemble Methods)

Algoritmo	Prós	Contras
Árvore de Decisão	Fácil de entender; interpretável	Pode facilmente sofrer de sobreajuste
Random Forest	Reduz o risco de sobreajuste; bom para dados complexos	Modelo mais difícil de interpretar; alto custo computacional
XGBoost	Alta precisão; eficiente em grandes volumes de dados	Requer ajuste fino de Hiperparâmetros; pode ser demorado de treinar
CatBoost	Suporta dados categóricos sem pré-processamento; ótimo desempenho	Mais complexo de configurar; pode ser lento com muitos dados

Árvore de Decisão (Decision Tree)

TEST DATA

DECISION TREE

Modelo para **classificação** e **regressão**.

◆ hierarquia baseada em nós, ramos e folhas.

```
# pip install scikit-learn
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Dados: Comprimento e largura das sépalas e pétalas
```

```
# X [sepal length, sepal width, petal length, petal width]
```

```
# y as espécies (0, 1, 2, correspondendo a diferentes espécies de Iris)
```

```
X = [[5.1, 3.5, 1.4, 0.2], # Iris-setosa  
      # ...  
      [5.8, 2.7, 5.1, 1.9]]
```

```
y = [0, ... , 2] # 0: Iris-setosa, 2: Iris-virginica
```

```
# Criar e treinar o modelo de Árvore de Decisão
```

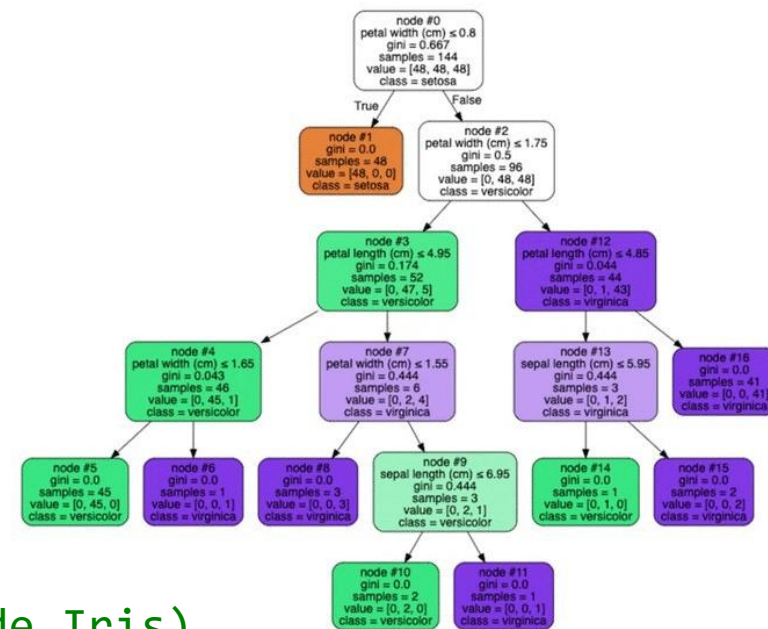
```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
# Fazer previsão para uma nova flor com sepal length, sepal width, petal length, petal width
```

```
new_flower = [[6.3, 3.3, 6.0, 2.5]] # nova flor
```

```
print(model.predict(new_flower)) # Saída será a classe prevista
```



Random Forest (Floresta Aleatória)

Algoritmo de Ensemble que **combina várias árvores de decisão**.

- Para classificação, usa a maioria das votações das árvores
- Para regressão, usa a média das previsões.

♦ **Reduz Sobreajuste**: Cria árvores independentes para melhores previsões

```
#pip install scikit-learn
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
# Dados de exemplo: Comprimento e largura de pétalas e sépalas
```

```
X = [[5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2], [6.3, 3.3, 6.0, 2.5], [5.8, 2.7, 5.1, 1.9]]
```

```
y = [0, 0, 2, 2] # 0: Iris-setosa, 2: Iris-virginica
```

```
# Criar e treinar o modelo Random Forest
```

```
model = RandomForestClassifier(n_estimators=100) # 100 árvores
```

```
model.fit(X, y)
```

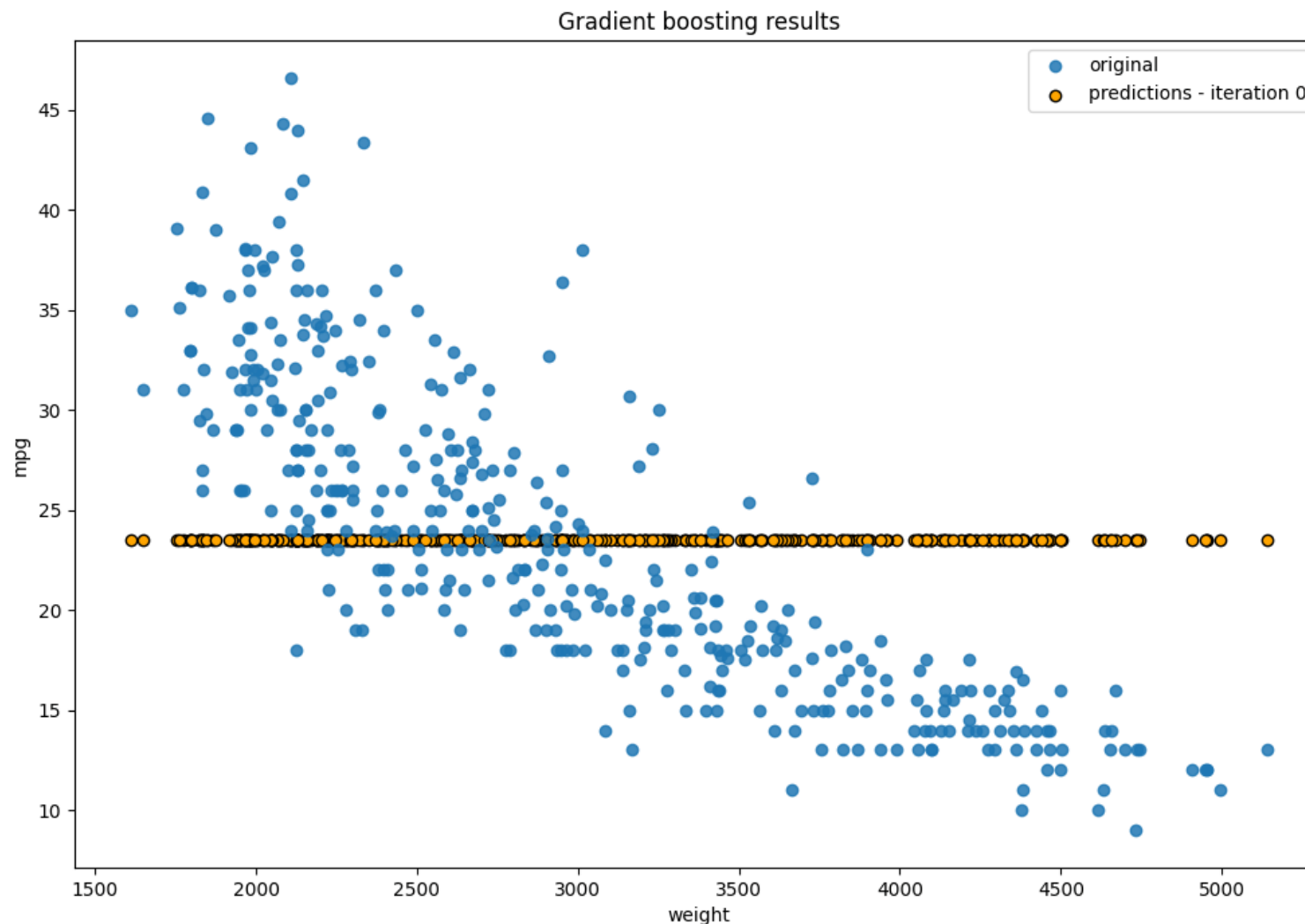
```
# Fazer previsão para uma nova flor
```

```
new_flower = [[6.3, 3.3, 6.0, 2.5]]
```

```
print(model.predict(new_flower)) # Saída será a classe prevista
```

Gradient Boosting

Gradient Boosting combina modelos fracos sequencialmente para melhorar a exactidão.



Baseado em Gradient Boosting, otimiza modelos de árvores de decisão em sequência, mas requer que os dados categóricos sejam pré-processados.

```
# pip install xgboost
```

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Dados de exemplo: Comprimento e largura de pétalas e sépalas
```

```
X = [[5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2], [6.3, 3.3, 6.0, 2.5], [5.8, 2.7, 5.1, 1.9]]
y = [0, 0, 2, 2] # 0: Iris-setosa, 2: Iris-virginica
```

```
# Dividir os dados em treino e teste
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Criar e treinar o modelo XGBoost
```

```
model = xgb.XGBClassifier()
model.fit(X_train, y_train)
```

```
# Fazer previsão
```

```
y_pred = model.predict(X_test)
print(f"Acurácia: {accuracy_score(y_test, y_pred)}")
```

CatBoost

Projetado especificamente para lidar com dados categóricos de forma nativa, **sem a necessidade de codificações de dados categóricos**.

```
# pip install catboost
```

```
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Dados de exemplo: Comprimento e largura de pétalas e sépalas
```

```
X = [[5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2], [6.3, 3.3, 6.0, 2.5], [5.8, 2.7, 5.1, 1.9]]
```

```
y = [0, 0, 2, 2] # 0: Iris-setosa, 2: Iris-virginica
```

```
# Dividir os dados em treino e teste
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Criar e treinar o modelo CatBoost
```

```
model = CatBoostClassifier(iterations=100, learning_rate=0.1, depth=5, verbose=0)
```

```
model.fit(X_train, y_train)
```

```
# Fazer previsão
```

```
y_pred = model.predict(X_test)
```

```
print(f"Acurácia: {accuracy_score(y_test, y_pred)}")
```

- ◆ Lab 06 - Tipos de aprendizagem - aprendizagem supervisionada
 - 6.4. - Métodos de Conjunto (Ensemble Methods) — Maximizar Desempenho e Explicar Previsões

Comparar algoritmos de Aprendizagem Profunda (Deep Learning)

Algoritmo	Prós	Contras
Redes Neurais Artificiais (ANN)	Versáteis; adequadas para diferentes tipos de problemas	Difíceis de interpretar; requerem muitos dados para treino
Redes Convolucionais (CNN)	Excelente para reconhecimento de imagens e padrões espaciais	Alto custo computacional; precisam de muitos dados rotulados
Redes Recorrentes (RNN)	Boas para dados sequenciais e séries temporais	Problemas de gradientes desaparecendo; difícil de treinar para sequências longas
Transformers + Atenção	Captura dependências de longo prazo; ótimo desempenho em NLP	Muito complexo e custoso; precisa de muitos dados e recursos para treinar

Redes Neurais Artificiais (ANN)

```
#pip install torch
import torch
import torch.nn as nn
import torch.optim as optim
```

```
# Definir o modelo ANN
```

```
class ANNModel(nn.Module):
```

```
    def __init__(self):
```

```
        super(ANNModel, self).__init__()
```

```
        self.fc1 = nn.Linear(4, 10) # 4 inputs (ex: features), 10 neurónios na camada oculta
```

```
        self.fc2 = nn.Linear(10, 1) # Camada de saída
```

```
    def forward(self, x):
```

```
        x = torch.relu(self.fc1(x)) # Ativação ReLU
```

```
        return torch.sigmoid(self.fc2(x)) # Ativação Sigmoid
```

```
# Criar o modelo
```

```
model = ANNModel()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
loss_fn = nn.BCELoss() # Função de perda para classificação binária
```

```
# Fazer previsão
```

```
X_new = torch.tensor([[5.1, 3.5, 1.4, 0.2]]) # Novos dados (exemplo)
```

```
model.eval() # Colocar o modelo em modo de avaliação
```

```
with torch.no_grad():
```

```
    prediction = model(X_new) # Prever a classe
```

```
    print(prediction.item()) # Resultado da previsão
```

Aprendizagem Supervisionada com Deep Learning

- Usado quer para **classificação** ou **regressão**

Redes Neurais Artificiais (ANN)

```
#pip install torch
import torch
import torch.nn as nn
import torch.optim as optim

# Definir o modelo ANN
class ANNModel(nn.Module):
    def __init__(self):
        super(ANNModel, self).__init__()
        self.fc1 = nn.Linear(4, 10) # 4 inputs (ex: features), 10 neurónios na camada oculta
        self.fc2 = nn.Linear(10, 1) # Camada de saída
    def forward(self, x):
        x = torch.relu(self.fc1(x)) # Ativação ReLU
        return torch.sigmoid(self.fc2(x)) # Ativação Sigmoid

# Criar o modelo
model = ANNModel()
optimizer = optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.BCELoss() # Função de perda para classificação binária

# Fazer previsão
X_new = torch.tensor([[5.1, 3.5, 1.4, 0.2]]) # Novos dados (exemplo)
model.eval() # Colocar o modelo em modo de avaliação
with torch.no_grad():
    prediction = model(X_new) # Prever a classe
    print(prediction.item()) # Resultado da previsão
```

Redes Convolucionais (CNN)

```
# pip install torch torchvision
import torch
import torch.nn as nn

# Definir o modelo CNN
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1) # Camada convolucional (1 input, 16 filtros)
        self.pool = nn.MaxPool2d(2, 2)           # Max Pooling
        self.fc1 = nn.Linear(16 * 14 * 14, 10)    # Camada totalmente conectada (para 10 classes)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # Convolução + ReLU + Pooling
        x = x.view(-1, 16 * 14 * 14)            # Flatten
        x = torch.softmax(self.fc1(x), dim=1)    # Camada de saída com softmax
        return x

# Criar o modelo
model = CNNModel()

# Exemplo de predição
model.eval() # Colocar o modelo em modo de avaliação
input_image = torch.randn(1, 1, 28, 28) # Exemplo de imagem de 28x28 com 1 canal (grayscale)
with torch.no_grad():
    prediction = model(input_image)
    print(prediction) # Predição de 10 classes
```

Redes Recorrentes (RNN)

Modelo Deep Learning
especializado em **dados
sequenciais**: Captura dependências
temporais e sequenciais

```
# pip install torch
```

```
import torch
import torch.nn as nn
```

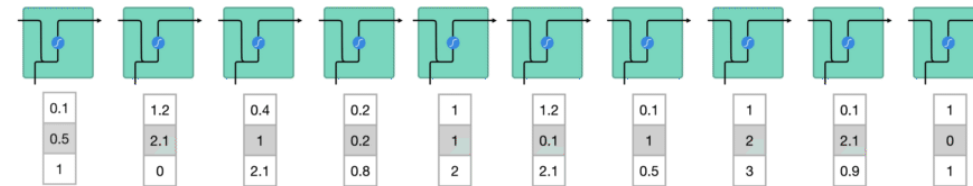
```
# Definir o modelo RNN
```

```
class RNNModel(nn.Module):
```

```
    def __init__(self, input_size, hidden_size, output_size):
        super(RNNModel, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)
```

```
    def forward(self, x):
```

```
        h0 = torch.zeros(1, x.size(0), hidden_size) # Estado inicial
        out, _ = self.rnn(x, h0) # Passagem pela RNN
        out = self.fc(out[:, -1, :]) # Camada totalmente conectada com a última saída
        return out
```



```
# Criar o modelo
```

```
input_size = 10 # Exemplo: 10 features de entrada
```

```
hidden_size = 20 # Tamanho do estado oculto
```

```
output_size = 1 # Exemplo: Saída para regressão
```

```
model = RNNModel(input_size, hidden_size, output_size)
```

```
# Exemplo de predição
```

```
model.eval()
```

```
# Exemplo de sequência (batch_size=1, sequence_length=5)
```

```
input_seq = torch.randn(1, 5, input_size)
```

```
with torch.no_grad():
```

```
    prediction = model(input_seq)
```

```
    print(prediction) # Saída da predição
```

Transformers + Atenção

- ◆ Modelo de Deep Learning especialmente eficiente para **dados sequenciais** e **processamento de linguagem natural** (NLP).

```
# pip install torch transformers
```

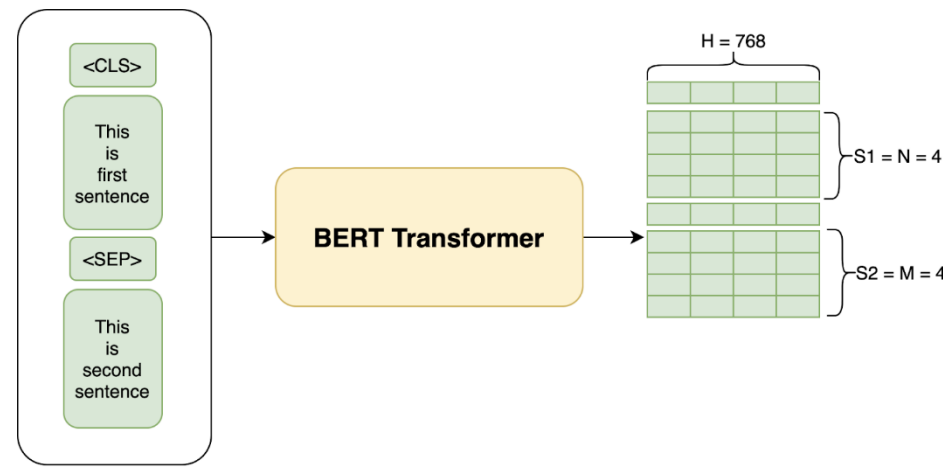
```
from transformers import BertTokenizer, BertModel
import torch
```

```
# Carregar modelo BERT pré-treinado e o tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
```

```
# Tokenizar o texto de exemplo
inputs = tokenizer("Este é um exemplo de texto.", return_tensors="pt")
```

```
# Fazer predição com atenção
with torch.no_grad():
    outputs = model(**inputs)
```

```
# Acesso à saída e aos embeddings do modelo
last_hidden_states = outputs.last_hidden_state
print(last_hidden_states)
```



- ◆ Lab 06 - Tipos de aprendizagem - aprendizagem supervisionada
 - 6.5 - Aprendizagem Profunda (Deep Learning) — Regressão com Rede Neural (1 Neurónio) — Estimar a Duração de Voo